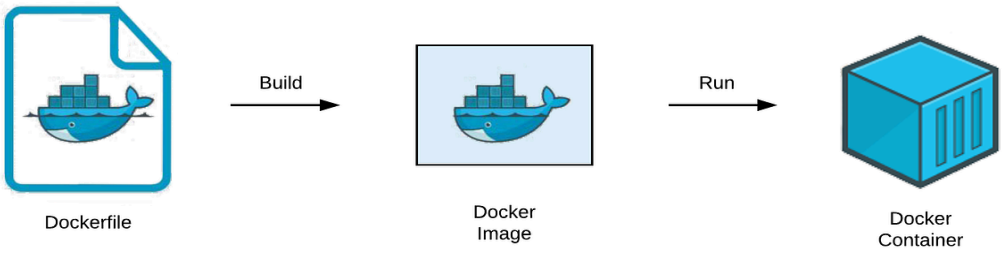
LAB MANUAL: INTRODUCTION TO DOCKER & CONTAINERIZATION

You do Full virtualization and para virtualization using virtual machines. But using containers you can do OS-Level Virtualization. Containers contain an application plus all its dependent libraries/environment. To make containers we use dockers. Dockers bundle up applications and its dependencies in a container.

How do Dockers make containers?
The Specifications of making a container are written in a file called a docker file.
Dockerfile is built to form a docker image. Docker image is a blueprint of a container. When you run a docker image it becomes a container.



DockerHub is an online repository where you get a collection of pre made docker images that you just need to pull and use without any hassle of setting up application dependencies.

TASKS DONE IN THIS LAB (to be done in exam)
- Creating a docker file
- Executing build command to create a docker image from a docker file
- Running docker image to make a container
- Pushing/uploading the image/container onto docker hub repository
- Pull any existing docker image from docker hub and run on your machine (owncloud image)

PART 1: IMPORTANCE OF DOCKERS

What is Docker?

Docker is an open-source platform that enables developers to build, deploy, run, update, and manage applications using **containers**.
A container is a lightweight, standalone, executable package that includes everything needed to run a piece of software, including the code, runtime, system tools, libraries, and settings.

Core Concepts

- **Docker Engine:** The core client-server application that installs on the host machine.
- **Docker Image:** A read-only template containing instructions for creating a Docker container.
- **Docker Container:** A runnable instance of a Docker image.
- **Docker Daemon (dockerd):** The background service managing Docker objects (images, containers, networks, volumes).
- **Docker Hub / Registry:** A repository service where Docker users can share and pull container images.

Why Docker is Important

- **Solves "It works on my machine":** Eliminates environment inconsistency between development, testing, and production environments.
- **Lightweight Efficiency:** Unlike traditional Virtual Machines (VMs), containers share the host OS kernel instead of virtualizing an entire operating system. This leads to faster boot times (seconds vs. minutes) and low resource overhead.
- **Microservices Architecture:** Allows developers to break down large applications into isolated, independent services running in separate containers.

PART 2: BASIC DOCKER COMMANDS

Command	Purpose	Explanation
docker --version	Verification	Displays the currently installed Docker version.
docker pull <image_name>	Download Image	Fetches a Docker image from Docker Hub to your local engine without running it.

docker images	List Images	Lists all Docker images currently stored locally on your machine.
docker ps	List Active Containers	Shows currently running containers and their details (Container ID, Ports, Status).
docker ps -a	List All Containers	Shows all containers, including stopped and exited ones.
docker run <image_name>	Run Container	Creates and starts a container from a specified image.
docker stop <container_id>	Stop Container	Gracefully shuts down a running container.
docker rm <container_id>	Delete Container	Removes a stopped container from the local storage.
docker rmi <image_id>	Delete Image	Deletes a local Docker image.

PART 3: CREATING, BUILDING, RUNNING & PUSHING A CUSTOM DOCKER IMAGE

1.Create the Application File: Step 1.
Create a working directory on your computer and navigate into it:

mkdir docker-lab && cd docker-lab

Create a simple HTML file named index.html:

```
HTML
<!DOCTYPE html>
<html>
<head>
   <title>Docker Lab</title>
</head>
<body>
   <h1>Hello! This application is running inside a custom Docker Container.</h1>
</body>
</html>
```

2.Write the Dockerfile: Step 2.
Create a file named Dockerfile (no file extension) in the same directory:

Dockerfile

```
# 1. BASE IMAGE: Use an official light Nginx web server image
FROM nginx:alpine

# 2. WORKDIR: Set working directory inside the container
WORKDIR /usr/share/nginx/html

# 3. COPY: Copy local index.html to container directory
COPY index.html .

# 4. EXPOSE: Document that container listens on port 80
EXPOSE 80

# 5. CMD: Start Nginx web server in foreground
CMD ["nginx", "-g", "daemon off;"]
```

Detailed Explanation of Dockerfile Instructions:

- **FROM**: Specifies the base image upon which you are building. nginx:alpine provides a lightweight Linux distribution with Nginx pre-installed.
- **WORKDIR**: Sets the working directory inside the container for subsequent instructions (COPY, RUN, CMD).
- **COPY**: Copies files from your local host directory into the container's filesystem.
- **EXPOSE**: Acts as documentation between the image author and the person running the container regarding which port the application listens on.

- **CMD**: Provides the default command executed when the container starts. daemon off; forces Nginx to stay in the foreground so the container does not exit immediately.

3.Build the Docker Image: Step 3.

Execute the build command. Replace your_dockerhub_username with your actual Docker Hub username:

docker build -t your_dockerhub_username/my-web-app:v1.0 .

- -t: Tags the image with a name and optional tag (name:tag).
- .: Specifies the current directory as the build context where the Dockerfile is located.

4.Run the Custom Container: Step 4.
Start a container using your newly built image:

docker run -d -p 8080:80 --name my-running-app your_dockerhub_username/my-web-app:v1.0

Flag Explanations:

- -d: Runs the container in **detached mode** (in the background).
- -p 8080:80: Port forwarding map (<Host_Port>:<Container_Port>). Maps port 8080 on your machine to port 80 inside the container.
- --name: Assigns a human-readable name to the container instance.

Test: Open your web browser and navigate to http://localhost:8080.

5.Push Image to Docker Hub: Step 5.
Log in to your Docker Hub account from your terminal:

docker login

Enter your Docker Hub credentials when prompted.

Push the built image to your repository:

docker push your_dockerhub_username/my-web-app:v1.0

PART 4: PULLING & RUNNING OWNCLOUD FROM DOCKER HUB

ownCloud is an open-source self-hosted file sync and content collaboration platform.

1.Pull the ownCloud Image: Step 1.
Fetch the official ownCloud server image from Docker Hub:

docker pull owncloud/server

Verify that the image was downloaded successfully:

docker images

2.Run the ownCloud Container: Step 2.
Start an instance of the ownCloud server using port mapping:

docker run -d -p 8080:8080 --name my-owncloud owncloud/server

- **-d**: Runs the container in the background.
- **-p 8080:8080**: Maps port 8080 on your host machine to port 8080 of the ownCloud instance.
- **--name my-owncloud**: Names the container instance my-owncloud.

3.Verify Container Status: Step 3.
Check if the container is running:

docker ps

To check container logs for initial boot steps:

docker logs my-owncloud

4.Access the ownCloud Interface: Step 4.

1. Open your web browser.
2. Navigate to http://localhost:8080.
3. You will be greeted by the ownCloud setup wizard where you can configure the administrator user account and finish setting up your private cloud storage instance.